

Лекция 6

RAG, который работает

chunking, retrieval, гибридный поиск, re-ranking и как всё это измерить

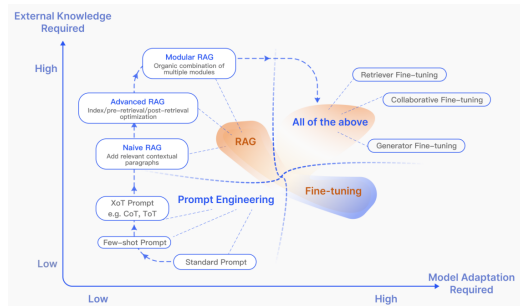
- ❶ **Зачем RAG**: дать модели знания, которых у неё нет
- ❷ **Chunking**: как резать документы, чтобы не сломать смысл
- ❸ **Retrieval**: поиск, гибриды, re-ranking
- ❹ **Измерять**: recall@k и почему без этого никак

Сборка недели

Память дала архитектуру, эмбединги дали поиск по смыслу. Сегодня собираем из этого RAG и учимся понимать, где он врёт.

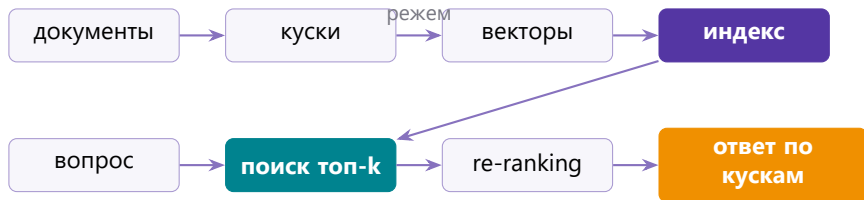
Зачем вообще RAG

- У модели отсечка знаний и нет ваших документов. Про внутренние правила и свежие данные она не знает
- RAG чинит это просто: находим нужные куски и **кладём их в контекст** вопроса
- Модель отвечает не из памяти, а из поданных документов. Бонус: можно **сослаться на источник** и проверить ответ
- Почему не дообучение: оно вшивает знания в веса, дорого и не обновить на лету. Для фактов почти всегда RAG, дообучение для стиля и формата



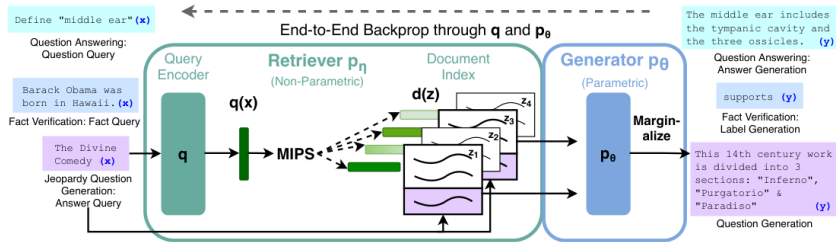
По горизонтали, сколько нужно менять модель. По вертикали, сколько нужно внешних знаний. RAG и дообучение не конкуренты, а разные оси

Источник: Gao et al., 2023. RAG survey



- Верхний ряд делается **один раз** при сборке: порезать, посчитать векторы, сложить в индекс
- Нижний ряд на каждый вопрос: найти близкие куски, переупорядочить, отдать модели
- Каждый узел этого конвейера умеет сломаться по-своему. Разберём по очереди

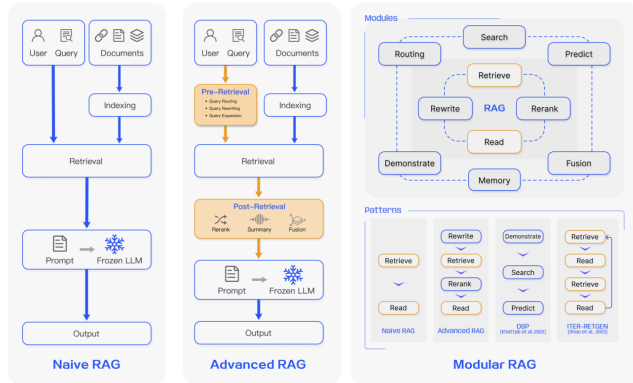
Откуда название: статья 2020 года



Источник: Lewis et al., 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

- В оригинале это одна обучаемая связка: энкодер запроса, индекс Википедии на 21 миллион кусков и генератор, которых учили вместе
- Прижилась не связка, а **схема**: любой поиск подкладывает куски любой модели через промпт. Ничего обучать не нужно
- Отсюда путаница в слове: сегодня RAG означает не конкретную модель, а весь конвейер с прошлого слайда

Наивный, продвинутый, модульный



- **Наивный:** порезали, нашли, отдали модели. Работает, но ломается на всём, о чём эта лекция
- **Продвинутый:** до поиска переписывают запрос, после поиска переупорядочивают и сжимают куски. Наша цель на сегодня
- **Модульный:** поиск, память, роутинг как отдельные блоки, которые собирают под задачу. К нему придём с агентами

- Документ целиком в индекс не кладут: искать надо **абзац по теме**, а не всю инструкцию на сто страниц
- Слишком крупный кусок: в контекст едет много лишнего, релевантное тонет
- Слишком мелкий кусок: смысл разорван, модель не соберёт ответ из обрывков
- Ищем золотую середину: кусок, который сам по себе отвечает на один вопрос

Ориентир

Абзац или несколько предложений, примерно 200–400 слов, с **перекрытием** соседних кусков, чтобы фраза на границе не потерялась. Точный размер подбирают под свой корпус и проверяют замером.

- Резать по N символов просто, но слепо: режет посреди предложения, посреди таблицы, посреди списка
- Классический провал: **таблица разорвана** между кусками. Заголовок в одном, строки в другом, ни один не отвечает на вопрос
- Лучше резать по структуре: по абзацам, заголовкам, разделам. Тогда кусок остаётся цельной мыслью
- Ещё лучше: добавить куску контекст, из какого документа и раздела он взят

Тариф	Завтрак	Выезд
Комфорт	да	14:00

кусок 1

Эконом	нет	12:00
--------	-----	-------

кусок 2

Заголовок таблицы оторван от строк: ни один кусок не отвечает целиком.

Перед нарезкой: разобрать документ по структуре



Источник: [docling-project/docling](https://github.com/docling-project/docling)

- PDF это картинка с буквами, а не текст. Заголовки, таблицы, колонки и сноски надо **распознать**, иначе резать нечего
- Парсеры вроде Docling и Unstructured отдадут документ как структуру: разделы, абзацы, таблицы целиком
- Резать по этой структуре, а не по символам, и есть защита от разорванной таблицы



Docling



Unstructured

Кусок без контекста: пример про Берлин

Berlin[a] is the capital and largest city of Germany, both by area and by population. [11] Its more than 3.85 million inhabitants [12] make it the European Union's most populous city, as measured by population within city limits. [13] The city is also one of the states of Germany, and is the third smallest state in the country in terms of area. Berlin is surrounded by the state of Brandenburg, and Brandenburg's capital Potsdam is nearby. The urban area of Berlin has a population of over 4.5 million and is therefore the most populous urban area in Germany. [5] [14] The Berlin-Brandenburg capital region has around 6.2 million inhabitants and is Germany's second-largest metropolitan region after the Rhine-Ruhr region, and the sixth-biggest metropolitan region by GDP in the European Union. [15]

Berlin[a] is the capital and largest city of Germany, both by area and by population. [11]

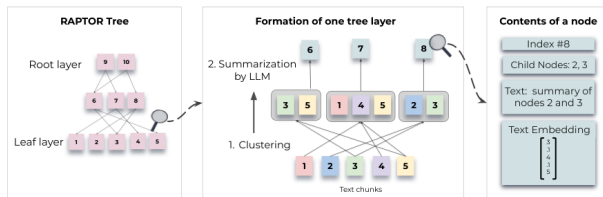
Its more than 3.85 million inhabitants [12] make it the European Union's most populous city, as measured by population within city limits. [13]

The city is also one of the states of Germany, and is the third smallest state in the country in terms of area.

Источник: Günther et al., 2024. Late Chunking

- Статья о Берлине порезана по предложениям. Во втором куске «его население 3,85 миллиона», в третьем «город входит в число земель». Слова **Берлин** там нет
- Эмбеddинг такого куска не знает, о чём он. Запрос «население Берлина» найдёт первый кусок, а не тот, где ответ
- Лечат тремя способами: приписать куску заголовок документа и раздела, отдавать в контекст родительский абзац, или считать эмбеddинг куска **с оглядкой на весь документ** (late chunking). В демо-агенте курса стоят первые два

Куски разного масштаба: parent-document и деревья



Источник: Sarthi et al., 2024. RAPTOR

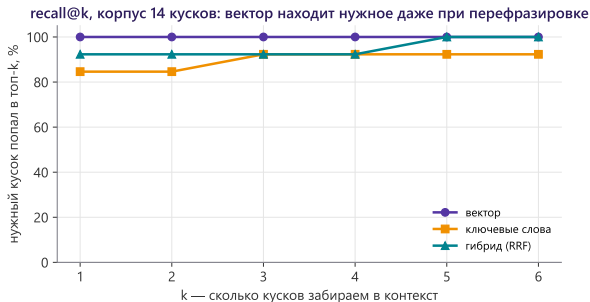
- **Parent-document retrieval:** ищем по мелким кускам, они точнее попадают в вопрос, а в контекст отдаём **родительский** абзац или раздел целиком
- **RAPTOR** идёт дальше: куски кластеризуют, каждую группу сжимают моделью в сводку, из сводок собирают дерево. Поиск идёт по всем уровням сразу
- Окупается на вопросах «о чём этот документ в целом», где ни один кусок не отвечает. Цена: сводки надо посчитать заранее

- Вопрос эмбеddят и ищут топ-k ближайших кусков. Это ровно поиск по смыслу с прошлой лекции
- Сколько брать: мало k, рискуем не захватить нужное. Много k, тащим шум и платим за длинный контекст
- Типичное k от 3 до 8, подбирается замером
- Дальше эти куски идут модели с инструкцией **отвечать только по ним**

Две половины ответа

Retrieval нашёл куски. Generation собрала ответ. Это **два разных** этапа, и ломаются они по-разному. Мерить их тоже надо порознь, об этом дальше.

Сколько кусков брать: recall@k



- recall@k это доля вопросов, для которых нужный кусок попал в первые k. Растёт с k, но и шум растёт
- Смысловый вектор находит нужное даже при полной перефразировке, где у запроса и куска нет общих слов
- По этой кривой и выбирают k: где recall выходит на плато, дальше увеличивать смысла мало

- Вектор ловит **смысл**, но спотыкается на точных строках: коды, артикулы, редкие имена, аббревиатуры
- Поиск по словам наоборот: смысл не понимает, но точное совпадение E-402 находит железно
- Гибрид считает оба и объединяет ранги, обычно через RRF: сложение обратных позиций в двух списках
- Идея: пусть каждый способ вытащит то, что умеет, а слияние соберёт лучшее

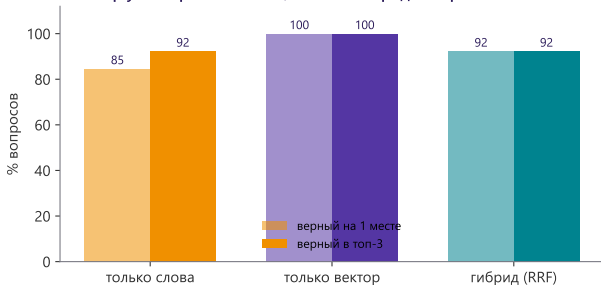
RRF в одной строке

$$\text{score}(d) = \sum_{\text{списки}} \frac{1}{K + \text{ранг}(d)}$$

Куску начисляют очки по позиции в каждом списке. Высоко в обоих значит высоко в итоге. Не нужно сводить несопоставимые близости к одной шкале.

Гибрид это не всегда лучше

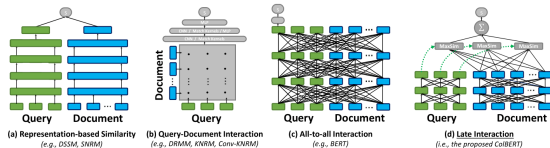
Сильный вектор уже берёт почти всё; слова и гибрид — страховка на точные термины



- На нашем чистом смысловом корпусе сильный вектор уже берёт почти всё, и гибрид его не обгоняет
- Гибрид окупается **на другом**: точные коды и термины, редкие слова, опечатки, большие корпуса
- Мораль недели: не добавляйте гибрид рефлексивно. Сначала замерьте вектор, потом решайте, нужна ли страховка

Re-ranking: второй, более умный взгляд

- Быстрый поиск даёт топ-k приблизительно: он сравнивает векторы, а не вникает в текст
 - Re-ranker берёт эти k кусков и переупорядочивает их
- внимательнее:** читает вопрос и кусок вместе
- Он точнее, но дороже, поэтому его пускают не на весь корпус, а только на найденные k
 - Окупается, когда нужный кусок находится, но стоит не первым. Если recall высок уже на малом k, можно обойтись без него



(а) наш векторный поиск: вопрос и кусок кодируют порознь. (с) cross-encoder, он же re-ranker: читает их вместе. (d) ColBERT посередине по цене и точности

Источник: Khattab, Zaharia, 2020. ColBERT

- Поиск отдал верные куски, вы проверили. Но агент всё равно отвечает неправильно
- Значит, сломалась **вторая** половина: generation. Куски есть, а модель их не так прочла
- Причины: противоречивые куски в контексте, нужное зарыто в середину (лекция про память), промпт не требует опираться только на источник
- Поэтому две половины меряют **порознь**: отдельно нашли ли, отдельно верно ли ответили

Faithfulness и correctness

Ответ бывает *верным по факту*, но не выведенным из поданных кусков: модель знала из претрейна. На закрытых данных это выстрелит. Проверяют не только правоту, но и **обоснованность** контекстом.

- Не на каждый вопрос ответ есть в корпусе. Честная система должна уметь сказать «не нашёл»
- Иначе агент вытащит **ближайший** кусок, даже если он не по теме, и уверенно ответит из него
- В промпт добавляют право отказаться, а в набор для проверки специально кладут вопросы без ответа
- Без неотвечаемых вопросов метрика всегда красивая, а на проде агент галлюцинирует на всём, чего нет в корпусе

Порог близости

Простая защита: если даже лучший кусок далеко от запроса, считаем, что ответа нет, и честно отказываемся. Порог подбирают замером.

Поиск

- **recall@k**: попал ли нужный кусок в топ-k
- **MRR**: насколько высоко он стоит в списке
- Меряют на наборе «вопрос и верный кусок»

Ответ

- **Correctness**: совпал ли по смыслу с эталоном
- **Faithfulness**: выведен ли из поданных кусков
- **Refusal**: честно ли отказался, когда ответа нет

Без замера RAG это гадание

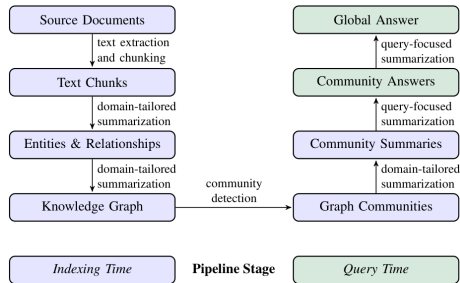
Улучшили chunking, поменяли k, добавили re-ranking. Стало лучше или хуже? На глаз не видно. Только фиксированный набор вопросов покажет, какая ручка что двигает. Полноценно этим займёмся на шестой неделе.

- В простом RAG поиск делается всегда: вопрос пришёл, нашли, ответили
- В агенте поиск это **инструмент**, который модель вызывает, когда сама решит, что ей не хватает знаний
- Тогда агент может искать несколько раз, переформулируя запрос, или не искать вовсе, если знает ответ
- Это соединяет три недели: инструмент (первая), память и знания (вторая), решение когда искать (агентный цикл)

Демо-агент курса

В нём `knowledge_base` это ровно такой инструмент: гибридный поиск с `parent-document retrieval`. Агент зовёт его сам, а сканер по дороге проверяет найденное на инъекции. Об этом на пятой неделе.

За горизонтом: RAG по графу знаний



Источник: Edge et al., 2024. From Local to Global: A Graph RAG Approach

- Обычный RAG отвечает на вопросы «где написано про X». На вопрос «какие главные темы во всём корпусе» ни один кусок не отвечает
- GraphRAG при индексации просит модель вытащить из кусков сущности и связи, строит граф и пишет сводки по его сообществам. Глобальный вопрос собирают из этих сводок
- Цена: индексация в десятки раз дороже, весь корпус проходит через модель. Брать, когда обычный RAG уперся



LlamaIndex



LangChain



Haystack



Docling



Unstructured



Cohere
Rerank



Elastic

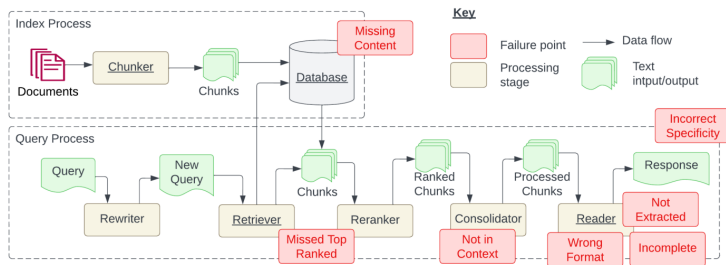


Ragas

- **Сборка конвейера:** LlamaIndex, LangChain, Haystack. Готовые загрузчики, нарезка, индексы, цепочки. Об этом на третьей неделе
- **Разбор документов:** Docling, Unstructured. PDF, DOCX, HTML в структуру с таблицами
- **Поиск и re-ranking:** Elastic для гибрида на больших корпусах, Cohere Rerank как готовый cross-encoder по API
- **Замер:** Ragas считает recall, faithfulness и correctness на вашем наборе вопросов

Всё это удобно, когда понимаешь, что внутри. Поэтому на семинаре сначала руками, а фреймворки со следующей недели.

Семь точек отказа глазами инженеров



Источник: Barnett et al., 2024. Seven Failure Points When Engineering a RAG System

- Авторы собрали три RAG-системы в проде и отметили, где именно они ломались. Красные блоки это наблюдения, а не гипотезы
- Половина отказов до генерации: нужного нет в корпусе, нашлось не в топе, не попало в контекст из-за лимита
- Вторая половина в генерации: ответ не извлечён из куса, не тот формат, не та детальность, неполный

- ❶ Наивный chunking по символам рвёт таблицы и предложения, и ни один кусок не отвечает целиком
- ❷ Кусок без контекста, из какого документа и раздела он взят: поиск находит, а смысл потерян
- ❸ Берут слишком большой k и топят нужное в шуме, или слишком маленький и не захватывают нужное
- ❹ Меряют RAG одним числом «верно или нет», не разделяя поиск и генерацию: непонятно, что чинить
- ❺ Нет неотвечаемых вопросов в наборе: метрика красивая, агент галлюцинирует на всём вне корпуса
- ❻ Добавляют гибрид и re-ranking рефлексивно, не проверив, что базовый вектор уже не справляется
- ❼ Правят chunking и k на глаз, без фиксированного набора вопросов

Руками

- RAG-агент поверх корпуса PDF: chunking, эмбединги, индекс
- Поиск топ-k и ответ по найденным кускам с указанием источника
- Замер recall@k на своём наборе вопросов
- Разбор провалов: где порвался кусок, где нашлось не то

Домашка

Собрать RAG-агента над своим корпусом и соединить с памятью прошлой недели. Дальше третья неделя: фреймворки и продвинутые паттерны поверх того, что вы собрали руками.

- 1 RAG кладёт нужные куски в контекст вопроса. Модель отвечает из документов, а не из памяти, и можно сослаться на источник
- 2 Chunking решает половину дела. Резать по структуре, не по символам, иначе рвётся смысл и таблицы
- 3 Вектор ловит смысл даже без общих слов. Гибрид и re-ranking это страховка, которую добавляют после замера, а не по умолчанию
- 4 Поиск и генерация ломаются порознь. Retrieval нашёл, а ответ неверный это про генерацию. Мерить надо обе половины
- 5 Без фиксированного набора вопросов, включая неотвечаемые, RAG это гадание. Замер показывает, какая ручка что двигает

Вопросы?

следующая неделя: фреймворки и продвинутые паттерны. LangGraph, smolagents, Reflection, Self-Consistency